

SaaS de Control Operativo de Transporte (MVP)

Versión: 1.0 (Web-Only First)	Fecha: Julio 2026	Plataforma: PWA / Responsive Web App
Enfoque: Control de Tiempos, Odómetro e Inspección	Arquitectura de Datos: PostgreSQL / Event Sourcing ligero	Target: Web Admin + Web Mobile Conductor

1. Visión del Producto y Premisas del MVP

El objetivo central de este MVP es proporcionar **visibilidad e integridad operativa** sobre el ciclo de vida de los viajes de transporte de carga y distribución, erradicando la dependencia de datos manuales no auditados.

Principios Inviolables del Desarrollo:

- **Enfoque Web-Only (Responsive / PWA):** No se desarrollarán aplicaciones nativas iOS/Android en esta etapa. El portal del conductor debe estar optimizado para navegadores móviles (Chrome/Safari móvil) con soporte PWA.
- **Verdad basada en evidencias:** Ninguna acción crítica (inicio/fin de viaje, discrepancias en inspección) se acepta solo con texto libre. Se requiere soporte fotográfico y geolocalización por API del navegador.
- **Auditoría Inmutable:** Los eventos del viaje (llegada, inicio descarga, fin descarga) se registran como append-only log con timestamp del servidor y coordenadas GPS.

2. Estrategia Técnica Web Mobile y Manejo Offline

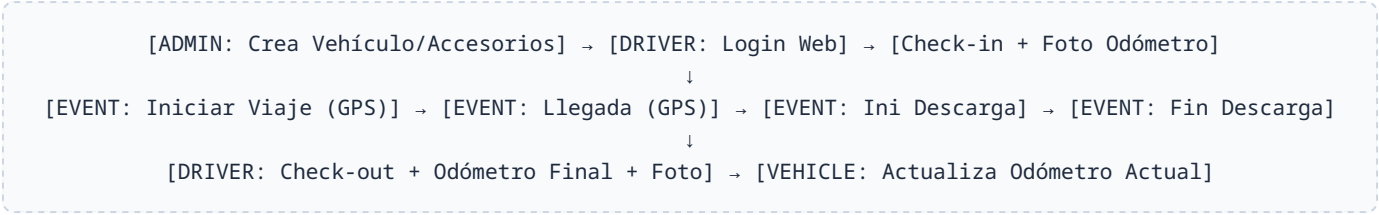
Dado que los conductores operarán a través de la Web en sus teléfonos inteligentes, el sistema debe abordar proactivamente la inestabilidad de la señal móvil en ruta:

- **Geolocalización vía Web API:** Uso estricto de `navigator.geolocation.getCurrentPosition()` al presionar botones de estado, capturando latitud, longitud y margen de precisión (accuracy).
- **Captura de Fotos HTML5:** Uso del atributo `<input type="file" accept="image/*" capture="environment">` para forzar la apertura directa de la cámara móvil sin subir archivos del carrete.
- **Manejo de Desconexión (PWA Offline First):** Cuando la red falle, los eventos (cambio de estado + coords GPS + timestamp local) se almacenan localmente en `IndexedDB` mediante un Service Worker. Al restablecer la conexión, la app sincroniza la cola de eventos indicando que fue un envío diferido.

3. Ciclo Operativo Paso a Paso (Driver Web App)

1. **Autenticación Simple:** Login mediante Usuario/Contraseña o PIN de 4 dígitos asignado por la empresa.
2. **Selección y Asignación de Unidad:** El conductor selecciona el vehículo asignado o escoge uno disponible.
3. **Inspección Previa (Check-in):**
 - Ingreso de odómetro inicial con **foto obligatoria del tablero**.
 - Verificación del checklist de accesorios preconfigurados (Gato, Extintor, Llave de ruedas, etc.).
 - Si un accesorio falta o presenta daño, la app exigen foto + descripción y genera una alerta administrativa.
4. **Ciclo de Registro de Viaje (Botones de Acción Inmutable):**
 - `[ Iniciar Viaje ]` → Captura GPS + Hora. Estado pasa a `IN_TRANSIT`.
 - `[ Llegada a Destino ]` → Captura GPS + Hora. Estado pasa a `AT_DESTINATION`.
 - `[ Iniciar Descarga ]` → Captura Hora. Estado pasa a `UNLOADING`.

- [Finalizar Descarga] → Captura Hora. Estado pasa a UNLOADING_COMPLETED .
- [Retorno / Fin de Viaje] → Solicita odómetro final + foto obligatoria del tablero. Estado pasa a COMPLETED .



4. Diseño de Base de Datos Relacional (PostgreSQL Schema)

El esquema garantiza aislamiento tenant mediante `company_id` y trazabilidad inmutable mediante eventos independientes.

```

-- 1. EMPRESAS (Multi-tenant Foundation)
CREATE TABLE companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. USUARIOS (Admin & Drivers)
CREATE TYPE user_role AS ENUM ('admin', 'driver');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  role user_role NOT NULL DEFAULT 'driver',
  pin_code VARCHAR(10),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. VEHÍCULOS
CREATE TYPE vehicle_status AS ENUM ('active', 'maintenance', 'inactive');

CREATE TABLE vehicles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  license_plate VARCHAR(20) NOT NULL,
  brand VARCHAR(100),
  model VARCHAR(100),
  current_odometer INT NOT NULL DEFAULT 0,
  status vehicle_status DEFAULT 'active',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_plate_per_company UNIQUE (company_id, license_plate)
);

-- 4. CATÁLOGO DE ACCESORIOS Y ASIGNACIÓN BASE
CREATE TABLE accessories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name VARCHAR(100) NOT NULL
);

CREATE TABLE vehicle_accessories (
  vehicle_id UUID REFERENCES vehicles(id) ON DELETE CASCADE,
  accessory_id UUID REFERENCES accessories(id) ON DELETE CASCADE,
  PRIMARY KEY (vehicle_id, accessory_id)
);

-- 5. VIAJES (Cabecera del Viaje)
CREATE TYPE trip_status AS ENUM (
  'created', 'inspected', 'in_transit', 'at_destination',
  'unloading', 'unloading_completed', 'completed', 'cancelled'
);

CREATE TABLE trips (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  vehicle_id UUID NOT NULL REFERENCES vehicles(id),
  driver_id UUID NOT NULL REFERENCES users(id),
  status trip_status NOT NULL DEFAULT 'created',

```

```

    start_odometer INT NOT NULL,
    end_odometer INT,
    start_odometer_photo_url VARCHAR(512) NOT NULL,
    end_odometer_photo_url VARCHAR(512),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP WITH TIME ZONE,
    CONSTRAINT check_odometers CHECK (end_odometer IS NULL OR end_odometer >= start_odometer)
);

-- 6. REGISTRO INMUTABLE DE EVENTOS (Append-Only Log)
CREATE TYPE trip_event_type AS ENUM (
    'start_trip', 'arrival_destination', 'start_unloading',
    'end_unloading', 'finish_trip'
);

CREATE TABLE trip_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trip_id UUID NOT NULL REFERENCES trips(id) ON DELETE CASCADE,
    event_type trip_event_type NOT NULL,
    recorded_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    gps_accuracy DECIMAL(6, 2),
    synced_offline BOOLEAN DEFAULT FALSE
);

-- 7. INSPECCIÓN PREVIA Y AUDITORÍA DE ACCESORIOS
CREATE TABLE trip_inspections (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trip_id UUID NOT NULL REFERENCES trips(id) ON DELETE CASCADE,
    accessory_id UUID NOT NULL REFERENCES accessories(id),
    is_present BOOLEAN NOT NULL DEFAULT TRUE,
    has_damage BOOLEAN NOT NULL DEFAULT FALSE,
    issue_description TEXT,
    issue_photo_url VARCHAR(512),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

5. Resumen de Tablas y Atributos Clave

Tabla	Claves	Campos Críticos	Propósito Operativo
companies	PK id	name, created_at	Aislamiento de clientes en arquitectura SaaS.
users	PK id FK company_id	email, password_hash, role, pin_code	Credenciales y roles (Admin / Conductor).
vehicles	PK id FK company_id	license_plate, current_odometer, status	Activo físico. Guarda el odómetro actualizado al finalizar viajes.
trips	PK id FK vehicle_id, driver_id	start_odometer, end_odometer, photos, status	Instancia del viaje con fotos de evidencia y auditoría de km.
trip_events	PK id FK trip_id	event_type, recorded_at, latitude, longitude	Log inmutable de auditoría con tiempos y GPS en cada hito.
trip_inspections	PK id FK trip_id, accessory_id	is_present, has_damage, issue_photo_url	Fotografía del estado de accesorios antes de arrancar.

6. Directivas Específicas de Prompt para Claude / AI Coder

Instrucciones para el Agente de Desarrollo (Claude):

Al procesar este documento para generar el código base del proyecto, debes seguir estas pautas estricta y secuencialmente:

- Stack Recomendado:** Next.js (App Router) / React para frontend web + Tailwind CSS; Supabase o PostgreSQL con Prisma/Drizzle ORM en backend.
- Fase 1 (Base de Datos y Auth):** Implementa el DDL en SQL proporcionado en la Sección 4. Configura políticas RLS (Row Level Security) o middleware para filtrar todas las consultas por `company_id`.
- Fase 2 (Web App Móvil Conductor):** Diseña la interfaz enfocada en pantalla de teléfono (390px width): botones grandes, contraste alto para luz solar, integración con Web Geolocation API y captura con cámara móvil.
- Fase 3 (Lógica de Negocio):** Crea un endpoint/transacción DB que al completar un viaje (`trip_status = 'completed'`), ejecute: `UPDATE vehicles SET current_odometer = new_end_odometer WHERE id = trip.vehicle_id;`
- Fase 4 (Panel Admin):** Dashboard web con vista de flota, tabla de viajes con tiempos calculados entre eventos (Tiempo en ruta = `arrival - start`; Tiempo de descarga = `end_unloading - start_unloading`).